

Chapter 1

프로그램 실행과 System Call 내부 구조

Ring 3 / Ring 0 · syscall · ELF · Dynamic Loader · execve → main
5 Lecture · 영상 87 분 · Ubuntu Server 26.04 + C

```
$ strace ./hello 2>&1 | wc -l
34
$ strace ./nostart
execve("./nostart", ...) = 0
write(1, "no main, no libc\n", 17)
exit(0)                      = ?
```

34 줄과 3 줄의 차이를 오늘 전부 설명한다

strace 34 줄을 전부 해석한다

L01

**User →
Kernel**

Ring 3 / Ring 0
Wrapper · errno

L02

syscall Entry

Register 규약
MSR_LSTAR · vDSO

L03

ELF

Section vs Segment
LOAD = mmap

L04

**Dynamic
Loader**

NEEDED · PLT/GOT
RELRO · PRELOAD

L05

**execve →
main**

auxv · _start
__libc_start_main

경계가 왜 있는가 → 어떻게 넘는가 → 실행 File 에는 무엇이 있는가 → 없는 코드를 어떻게 호출하는가 → 전부 이어 붙이기

실습 폴더 ~/lsp2/ch01/src/L01 ~ L05 · 빌드 make · 도구 strace ltrace gdb readelf objdump
자료 lecture.html (빈칸 · 괄호 42 칸) · animation.html (5 종) · worksheet.html (50 문항)

Application 에서 Kernel 까지 , 실제로 일어나는 일

Ch1

실행 · System Call

Ch2

Virtual Memory

Ch3

**Page Cache · File
I/O**

Ch4

Atomic · futex

Ch5

epoll · eventfd

Ch6

io_uring

Ch7

**strace · perf ·
eBPF**

Ch8

Isolation · Final

최종 도착점 : timerfd · signalfd · eventfd · inotify 를 하나의 epoll Loop 로 처리하는 Linux Event & Process Monitor 를 만들고 , strace · perf · ftrace · bpftrace 로 그 동작을 설명한다 .

Kernel 은 이름이 아니라 번호만 안다

printf()

Library 함수
문자열 생성 · Buffer

>

write()

glibc Wrapper
"1 번, 인자 3 개" 준비

>

syscall 명령

```
rax=1 rdi=1  
rsi=msg rdx=14
```

>

Kernel

write 처리 함수
(__x64_sys_write)

```
$ ./hello3_syscall  
hello, kernel  
SYS_write = 1
```

```
/* hello3_syscall.c */  
syscall(SYS_write, STDOUT_FILENO,  
        msg, sizeof(msg) - 1);
```

write() 는 Kernel 함수가 아니다. 번호 1 과 인자를 Register 에 싣고 Kernel 에 진입해 주는 glibc 의 작은 함수 (Wrapper) 다.

Kernel 에 가는 일 , 가지 않는 일

+0

strlen 300 만 번 호출

libonly: strace -c total 28 (hello2_write
29)

하려는 일	Kernel	이유
문자열 길이 , 정렬 , 계산	아니오	내 Memory 안에서 끝남
화면 출력 , File 읽기 / 쓰기	예	장치는 모든 Process 가 공유
Memory 더 받기 (brk, mmap)	예	Physical Memory 는 공유 자원
Process 생성 , Signal	예	다른 Process 에 영향
현재 시각 읽기	?	L02 에서 반전 (vDSO)

기준은 하나 : **내 Process 바깥의 자원을 건드리는가**

strace = Kernel 과의 경계 · ltrace = Library 와의 경계 (printf 가 puts 로 보이는 이유 : gcc 최적화)

Ring 3 에서 특권 명령을 실행하면

Ring 3 · User Mode

Application, glibc

특권 명령 (hlt, cli, I/O Port) 불가 → Exception

자기 Process 의 User 영역 Memory 만

장치 접근 불가

Ring 0 · Kernel Mode

Linux Kernel

특권 명령 가능

전체 Memory

장치 접근 가능

```
$ ./priv
```

```
Segmentation fault (core dumped)
```

```
traps: priv[2417] general protection fault
```

Ring 0 로 가는 길은 셋뿐
System Call · Exception · Interrupt

▶ 애니메이션 [C1-1 User ↔ Kernel Mode](#)

Kernel 은 음수를 , glibc 는 -1 과 errno 를 준다

Kernel

```
return -EBADF
rax = -9
```

>

glibc Wrapper

```
-4095 ~ -1 이면
errno = 9
return -1
```

>

main()

```
ret == -1 ?
strerror(errno)
```

**규칙 : 반환값이 실패 (-1) 일 때만
errno 를 읽는다**

```
$ strace -e trace=write ./errno_demo
write(99, "x", 1) = -1 EBADF (Bad file descriptor)
return = -1
errno = 9 (Bad file descriptor)
```

Kernel 에는 errno 변수가 없다 . errno 는 Thread 마다 있는 User Space 변수 (TLS). -9 가 그대로 오는 것은 L02 에서 Wrapper 없이 확인한다 .

Stack 이 아니라 정해진 Register 에 실는다

Register	역할	write(1, msg, 14)
rax	System Call 번호	1
rdi	인자 1	1 (fd)
rsi	인자 2	msg 주소
rdx	인자 3	14
r10 · r8 · r9	인자 4 · 5 · 6	
rax (복귀 후)	결과 · 오류면 -errno	14
rcx · r11	CPU 가 덮어씀	복귀 주소 · RFLAGS

왜 네 번째는 r10 인가

일반 함수 : rdi rsi rdx rcx r8 r9

System Call: rdi rsi rdx r10 r8 r9

syscall 명령이 rcx 에 복귀 주소를 써 버리기 때문

```
$ grep -w -E "__NR_(read|write|getpid|exit)" \
/usr/include/x86_64-linux-gnu/asm/unistd_64.h
#define __NR_read 0
#define __NR_write 1
#define __NR_getpid 39
#define __NR_exit 60
```

명령 하나가 CPU Hardware 로 한 번에 하는 일

rcx ← rip

돌아올 주소 보관

r11 ← rflags

Flag 보관

rip ← MSR_LSTAR

단 하나의 입구로

CPL 3 → 0

Ring 0 전환

entry_SYSCALL_64 Kernel Stack 전환, Register → pt_regs

↓

do_syscall_64 → x64_sys_call (switch) 번호로 분기

↓

__x64_sys_write → ksys_write → vfs_write

↓

regs->ax = 결과 → sysretq (rip ← rcx, Ring 3)

gdb catch syscall 로 멈추면 rax 는 -38(-ENOSYS) 이고 원래 번호는 \$orig_rax 에 있다.

Kernel 6.9+ x86-64 는 sys_call_table 함수 포인터 대신 switch 로 분기한다 (Spectre 완화).

▶ 애니메이션 [C1-2 System Call Entry](#)

Kernel 에 가지 않는 System Call

28.7 ns

clock_gettime() · vDSO

[vdso] 함수를 일반 call · [vvar] 읽기

166 ns

syscall(SYS_clock_gettime)

Mode·Stack 전환 , pt_regs, 보안 완화

5.8×

같은 일의 비용 차이

VirtualBox 에서는 더 벌어질 수 있다

```
$ strace -c -e trace=clock_gettime ./vdso_time
100.00  0.412345  0  1000000  clock_gettime  ← 200 만이 아니라 100 만
```

이 고정 비용이 이후 **Chapter** 의 동기다 : **stdio Buffer · Page Cache (Ch3) · futex (Ch4) · epoll (Ch5) · io_uring (Ch6)**

실행 File 하나에 목차가 두 개

ELF Header (64 Byte)

목차의 목차 · Magic 7f 45 4c 46 · Entry point

Program Header Table

" 실행할 때 이렇게 Memory 에 올려라 " ←
Kernel, ld.so

.text .rodata .data ... 실제 내용

Section Header Table

" 내용은 이런 조각으로 나뉘어 있다 " ← ld, gdb,
readelf

```
$ readelf -h sections
```

```
Type: DYN (Position-Independent  
Executable file)
```

```
Entry point address: 0x1080
```

```
Number of program headers: 13
```

```
Number of section headers: 37
```

**Entry point 는 main 이 아니다 .
처음 실행되는 곳은 _start (L05)**

변수는 어디에 들어가는가

Section	들어가는 것	Flag
<code>.text</code>	함수의 기계어	AX
<code>.rodata</code>	const 전역, 문자열 Literal	A
<code>.data</code>	초기값이 있는 전역 / static	WA
<code>.bss</code>	초기값 없는 (0) 전역 / static	WA · NOBITS
(없음)	지역 변수 — 실행 중 Stack	

0 이 아닌 값이 하나라도 있으면 배열 전체가 `.data` 로 간다. 임베디드 · Container Image 에서 File 크기를 좌우한다.

17 KB

big[10MB];

.bss — 크기만 기록

10 MB

big[] = {1};

.data — 내용이 File 에

`g_str` 는 `.data`, 그것이 가리키는 "string literal" 은 `.rodata`
 → `g_str[0] = 'S';` 는 Compile 되지만 실행하면
 Segmentation fault

LOAD 한 줄은 Kernel 에게 내리는 mmap 지시서

Type	Offset	VirtAddr	FileSiz	MemSiz	Flg
LOAD	0x000000	0x0000	0x0006a0	0x0006a0	R
LOAD	0x001000	0x1000	0x0002ad	0x0002ad	R E
LOAD	0x002000	0x2000	0x0001d4	0x0001d4	R
LOAD	0x002db0	0x3db0	0x000270	0x0012b0	RW

02 · R Loader 정보

03 · R E .text — 쓰기 금지

04 · R .rodata

05 · RW .data .bss .got — 실행 금지

Section 37 개 → LOAD 4 개

권한이 같은 것끼리 Page(4KB) 단위로 묶는다. W^X.

MemSiz - FileSiz = 0x1040

= .bss. File 에는 없고 Memory 에만 0 으로

▶ 애니메이션 [C1-3 ELF Explorer](#)

같은 Source, 세 가지 실행 File

PIE (기본)

Type DYN

0x1080 상대 주소
실행마다 시작 주소가 바뀜 (ASLR)

INTERP 있음 · 17,840 Byte

non-PIE (-no-pie)

Type EXEC

0x401070 고정 주소
관찰용 (L04 GOT)

INTERP 있음 · 17,768 Byte

Static (-static)

Type EXEC

0x401740 고정 주소
libc 코드가 File 안에

INTERP 없음 · 787,112 Byte

```
$ ./sections | head -1; ./sections | head -1
.text func 0x55f0a8c3b169
.text func 0x562b3d1f4169    ← 끝자리 169 만 같다
```

Library 는 이름만 기록된다

```
$ ./main
error while loading shared libraries:
libgreet.so: cannot open shared object file
$ readelf -d main | grep NEEDED
(NEEDED) Shared library: [libgreet.so]
(NEEDED) Shared library: [libc.so.6]
```

1 **LD_LIBRARY_PATH** 개발 · Test. setuid 에서 무시

2 **RUNPATH** -Wl,-rpath,'\$ORIGIN'

3 **/etc/ld.so.cache** ldconfig 가 만든 색인

4 **기본 경로** /lib, /usr/lib

-L. 은 Link 할 때 ld 에게만 쓰이고 실행 File 에 남지 않는다 . 실행할 때 이름을 실제 File 로 바꾸는 것은 Dynamic Loader (ld-linux-x86-64.so.2 = INTERP) 의 첫 번째 일이다 .

없는 코드를 어떻게 호출하는가



```

(gdb) x/gx 0x403340      # 첫 호출 전
0x403340 <greet@got.plt>: 0x000000000000401050
(gdb) next              # 첫 호출 후
0x403340 <greet@got.plt>: 0x00007ffff7fb9119
  
```

코드는 읽기 전용이라 고칠 수 없다

→ 고칠 수 있는 Data 영역에 주소 칸 (GOT) 을 두고
코드 (PLT) 는 그 칸을 거친다.
끝자리 119 = nm -D 의 greet Offset 0x1119

▶ 애니메이션 C1-4 PLT / GOT

최신 Ubuntu 는 왜 Lazy Binding 을 쓰지 않나

Lazy Binding (-z lazy)

함수별 첫 호출 때 GOT 를 채운다
시작이 빠르다
실행 중 GOT 가 쓰기 가능
→ GOT Overwrite 공격 가능

Full RELRO (-z now -z relro)

main 이전에 GOT 를 모두 채운다
시작이 조금 느리다
다 채운 뒤 mprotect 로 읽기 전용
→ 공격 차단 · Ubuntu gcc 기본

```
$ readelf -d main_rpath | grep FLAGS
(FLAGS)      BIND_NOW
(FLAGS_1)    Flags: NOW PIE
```

ltrace 가 비어 있는 이유도 이것 . 이 과정의 Makefile 이 ltrace 실행 대상에 -Wl,-z,lazy 를 붙인다 .

Symbol 검색 순서로 함수를 가로챈다

```
$ ./dice
rolling...
2 3 4 1 4
$ LD_PRELOAD=./libhook.so ./dice
[hook] puts("rolling...")
rolling...
6 6 6 6 6
```

방법	하는 일
rand() 바꿔치기	return 5; → $5 \% 6 + 1 = 6$
puts() 감싸기	Log 후 dlsym(RTLD_NEXT, "puts")
dlopen / dlsym	실행 중에 Library Load (Plugin)

효과 없음 : Static 실행 File · syscall 명령을 직접 실행하는 코드 (raw_syscall) · setuid 프로그램

dice 를 다시 Build 하지 않았다 . Loader 는 Load 된 순서대로 Symbol 을 찾고 LD_PRELOAD Library 가 가장 먼저 Load 된다 .

main 은 29 번째 줄에서야 시작된다

1	Kernel	execve: 실행 File 과 ld.so 를 Mapping, Stack 구성
2-8	ld.so	자기 Memory, ld.so.preload, ld.so.cache 로 libc 경로 찾기
9-18	ld.so	libc 의 ELF Header 읽기 → LOAD Segment 를 하나씩 mmap
19-28	ld.so / libc	TLS (arch_prctl) · RELRO mprotect · cache munmap
29-33	main	printf → fstat · brk · write(1, "hello\n", 6)
34	libc	exit_group(0)

14 번 mmap(..., PROT_READ|PROT_EXEC, ..., 0x28000) = libc 의 LOAD 한 줄 (Offset 0x028000, R E) — L03 의 지시서가 그대로 실행된다 .

execve 한 줄 안에서 Kernel 이 하는 일

```

1 File 앞부분 읽기 : \177ELF → load_elf_binary
2 Program Header 읽기, PT_INTERP → ld.so 열기
— point of no return —
3 기존 주소 공간 폐기 (fd, PID 는 유지)
4 PT_LOAD 를 mmap (.bss 는 0 Page)
5 ld.so 의 PT_LOAD 도 mmap
6 새 Stack: argc argv envp auxv · vDSO
7 rip = ld.so Entry → "복귀"

```

문자열 `"/hello"` `"PATH=..."`

`auxv` `AT_PHDR` `AT_ENTRY` `AT_BASE` ...

`envp[]` · `NULL`

`argv[]` · `NULL`

`argc` ← `rsp`

높은 주소 ↑ · 낮은 주소 ↓

ELF 를 읽어 들이는 것이 아니라 `mmap` 으로 대응시킨다 . 500MB 실행 File 도 `execve` 는 즉시 끝난다 (Demand Paging → Chapter 2).

main 은 `__libc_start_main` 이 부르는 Callback



```
(gdb) starti
0x00007ffff7fe4540 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) backtrace          # break main 후
#0 main () at hello.c:6
#2 __libc_start_main_impl (main=0x55555555149 <main>, ...)
#3 _start ()
```

Kernel 이 처음 실행시키는 코드는 `main` 이 아니라 Dynamic Loader 의 `_start`.

`exit()` 가 `stdio` Buffer 를 `Flush` 하는 곳 — 기존 과정의 `exit` vs `_exit`.

▶ 애니메이션 [C1-5 execve → main](#)

Kernel 이 아는 것은 Entry Point 하나

34

Dynamic hello

strace 줄 수 · 17 KB

13

Static hello

Library Mapping 없음 · 786 KB

3

nostart

execve · write · exit · 10 KB

```
void _start(void)
{
    sys3(1, 1, msg, 17);
    sys3(60, 0, 0, 0);
}
```

	Dynamic (기본)	Static (-static)
처음 점프하는 곳	ld.so 의 _start	실행 File 의 _start
libc 보안 Update	libc 만 교체	다시 Build
LD_PRELOAD · ltrace	동작	동작 안 함

7 개 개념 , 한 줄씩

개념	핵심 한 줄
Ring 3 / Ring 0	CPU Hardware 가 강제하는 경계 . 넘는 길은 System Call · Exception · Interrupt
Wrapper · errno	write() 는 번호를 싣는 glibc 함수 . Kernel 의 음수를 -1 과 errno 로 바꾼다
syscall 명령	rcx·r11 저장 , MSR_LSTAR 로 점프 , CPL 3→0 을 한 번에
vDSO	Kernel 진입 없이 읽기만 하는 System Call. 비용 차이 5 배 이상
Section / Segment	Section 은 Linker 용 , Segment 는 Kernel 용 . LOAD = mmap 지시서
PLT / GOT	없는 코드를 GOT 칸을 거쳐 호출 . Ubuntu 는 BIND_NOW + RELRO
execve → main	Kernel → ld.so _start → _start → __libc_start_main → main → exit

Virtual Memory 와 Memory Management 심화

오늘 두 번 미뤄 둔 이야기 : "mmap 은 대응 관계만 기록한다 ", " 실제 내용은 처음 접근할 때 올라온다 ". 주소 공간이란 무엇이고 , 접근하는 순간 무슨 일이 일어나는가 — Page Table · TLB · Page Fault · mmap.

개념 워크시트 50 문항

[worksheet.html](#)

Lecture 당 10 문항 · 채점 · 오답 정리

애니메이션 5 종

[animation.html](#)

C1-1 ~ C1-5

강의 자료 · 빈칸 42 칸

[lecture.html](#)

빈칸 채우기 5 · 괄호 넣기 5

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog